

Archivos (texto, binarios, json)

Clase 15

Archivos

Los archivos se utilizan para almacenar datos de manera **persistente**, lo que significa que los datos pueden recuperarse incluso después de que el programa que los creó se haya cerrado.

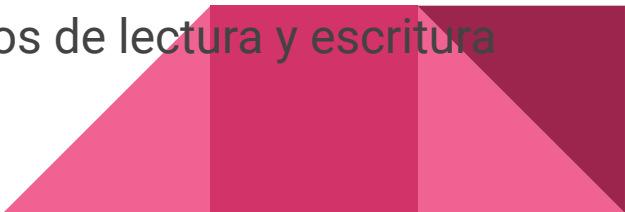


Archivos de texto

Son archivos que almacenan datos en forma de texto plano, como cadenas de caracteres. Pueden ser editados con cualquier editor de texto y se pueden leer y escribir con funciones como `open()`, `read()`, `write()`, entre otras.

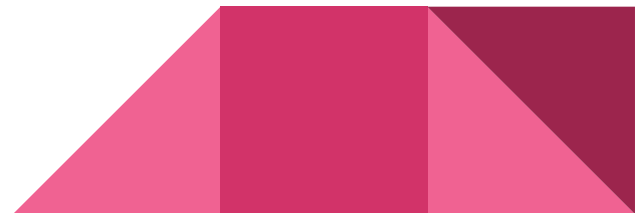


Modos de lectura

- 'r' (read): modo de lectura **por defecto**. Permite leer el archivo.
 - 'w' (write): modo de escritura. Permite escribir en el archivo, eliminando su contenido anterior.
 - 'x' (exclusive): modo de escritura exclusivo. Permite escribir en el archivo solo si este no existe. Si existe, lanza una excepción.
 - 'a' (append): modo de escritura en el que se agrega contenido al final del archivo sin eliminar su contenido anterior.
 - 'b' (binary): modo binario. Se utiliza junto con los modos de lectura y escritura para leer y escribir archivos binarios.
 - 't' (text): modo de texto. Se utiliza junto con los modos de lectura y escritura para leer y escribir archivos de texto.
- 

Modos de lectura (algunas combinaciones)

- lectura: 'rt', 'rb'
- escritura: 'wt', 'wb'
- agregado: 'at', 'ab'
- lectura/escritura: 'w+', 'wt+', 'wb+'
- agregado/lectura: 'a+', 'at+', 'ab+'



Formas de abrir un archivo

Un archivo abierto debe ser cerrado al terminar de procesarlo ya que pueden provocar inconsistencias.

```
# codigo
```

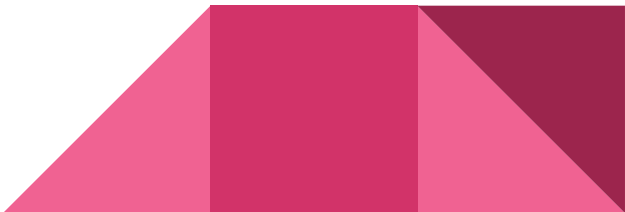
```
f = open('archivo.txt', 'w+')  
contenido = f.read()  
contenido += '\nFirma:'  
f.write(contenido)  
f.close()
```

```
# codigo
```

```
# codigo
```

```
with open('archivo.txt', 'w+') as f:  
    contenido = f.read()  
    contenido += '\nFirma:'  
    f.write(contenido)
```

```
# codigo
```



Métodos

- `archivo.close()` # Cierra el archivo.
 - `archivo.read()` # Retorna todo el contenido del archivo
 - `archivo.read(cantidad)` # Retorna una porción del contenido
 - `archivo.readlines()` # Retorna una lista con las cadenas del archivo como elementos separados por salto de linea
 - `archivo.seek(pos)` # Te redirige a una posicion del archivo
 - `archivo.tell()` # Retorna la posicion del archivo en la que te encuentras
 - `archivo.write(cadena)` # Escribe cadena en el archivo y retorna cantidad de caracteres escritos.
- 

Ejemplo

```
def leo_caracteres(nombre: str):  
    with open(nombre, "r") as f:  
        for x in f.read():  
            print(x)
```

```
def leo_lineas(nombre: str):  
    with open(nombre, "r") as f:  
        print(f.readlines())
```

```
def otra_forma(nombre: str):  
    f = open(nombre, "r")  
    for linea in f:  
        print(linea)  
    f.close()
```

```
def main():  
    print('Leo caracteres')  
    leo_caracteres()  
    print('-' * 100)  
    print('Leo lineas')  
    leo_lineas()  
    print('-' * 100)  
    print('Otra forma')  
    otra_forma()
```

```
if __name__ == '__main__':  
    main()
```



Archivos binarios (pickle)

El módulo pickle de Python es usado para serializar y deserializar todo tipo de datos (objetos) propios del lenguaje en binarios.

- `pickle.dump(object, archivo)` # Serializa cualquier objeto en el archivo binario
- `pickle.dumps(obj)` # Retorna en binario el objeto recibido
- `pickle.load(archivo)` # Deserializa los datos de un archivo
- `pickle.loads(obj)` # Deserealiza el binario del objeto recibido



Ejemplo

```
import pickle
```

```
def suma():  
    return 1+2
```

```
with open('archivo.dat', 'wb') as f:  
    pickle.dump(suma, f)
```

```
with open('archivo.dat', 'rb') as f:  
    fun = pickle.load(f)  
    print(fun)
```

```
binario = pickle.dumps(suma))  
# print('dumps', binario)  
# print('tipo: ', binario)  
# print('loads ', pickle.loads(binario))
```



JSON (JavaScript Object Notation)

```
{  
  "age": 30,  
  "city": "New York",  
  "hasChildren": false,  
  "name": "John",  
  "titles": [  
    "engineer",  
    "programmer"  
  ]  
}
```

Python	JSON
dict	object
list, tuple	array
str	string
int, long, float	number
True	true
False	false
None	null

JSON (JavaScript Object Notation)

- `json.load(file)`: Lee datos json de un archivo
- `json.loads(cadena)`: Lee datos json de un string
- `json.dump(json, file)`: Escribe un objeto json en un archivo
- `json.dumps(json)`: Retorna un string del objeto json recibido



Ejemplo api

```
import json
from urllib.request import urlopen

# https://apis.datos.gob.ar/series/api/series?start_date=2018-08&ids=168.1_T_CAMBIOR_D_0_0_26
response = urlopen("https://nolaborables.com.ar/API/v2/feriados/2023")

api = json.loads(response.read())

print(api)
```



Archivos en Python

Texto	Json	Pickle
• Solo maneja strings. • No hay necesidad de importar ningún módulo. • Brinda más funcionalidades por defecto.	• Solo maneja objetos compatibles con JavaScript • Es la notación más usada en la web. • Es compatible con muchos sistemas.	• Maneja cualquier tipo de objeto. • Los archivos creados solo pueden ser deserializados por Python. • Se debe confiar en el contenido del archivo.



Ejercicio 1

Crear un nuevo archivo 'neruda2.txt', el cual contendrá una nueva versión del archivo 'neruda.txt'. La nueva versión consiste en modificar la primera letra de cada palabra del archivo original a mayúscula.



Ejercicio 2

Agregar al programa “Ensalada de letras” la funcionalidad de guardar el puntaje más alto de cada usuario. Para esto se deberá pedir el nickname del jugador al empezar el juego. El puntaje y nombre serán guardados en un archivo json.

